

슈퍼 AAAAAAA급 게임

원하시는  
>> 명해주세요.

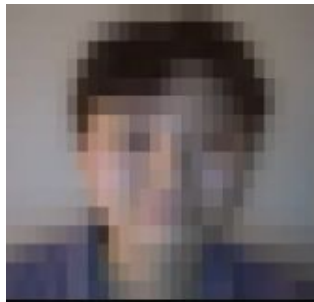
(구)쥬넨10조  
쥬넨~10~어렵조

## 팀원 소개



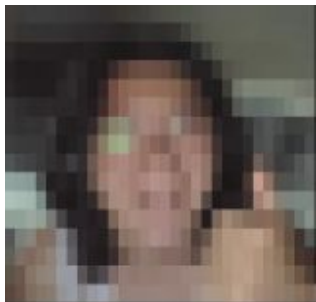
팀장 막말의 김진우

“그것도 몰라요?”



팀원 뽕대加里 조민솔

“저는 뽕대加里예요 ㅠㅠ”



팀원 배큐맨 임현우

10조(쵸낸10조) 지은이  
팀 공식 치어리더



팀원 밝히는 박찬형

“분명 치어리더라고 했는데”



# 게임 소개 : Super EZ RPG

나는 경이로운 AAAAAAA급 시네마틱 영상을 만들었지만,

I'm so sorry. I forgot i have to prepare our game play video for this presentation. Actually, i don't know how can i make the video. Preparing the video makes me bother.

용량이 너무 커서 여기에 업로드하지는 않겠다.

Instead, I'll show you playing game myself with the Fermat Meme.



피에르 드 페르마  
(*Pierre de Fermat, 1601~1665*)

# 핵심기능

- Scene System
- Img2Ascii
- SoundPlayer
- Combat System
- Mercenary System
- TargetingValidator
  - Skill
  - Item
  - AI



# 핵심기능 : SceneSystem

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        IScene currentScene = new CreateCharacterScene();
        while (true)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Console.Clear();
            currentScene.OnShow();
            currentScene = currentScene.GetNextScene();
            if (currentScene == null)
            {
                break;
            }
        }
    }
}
```





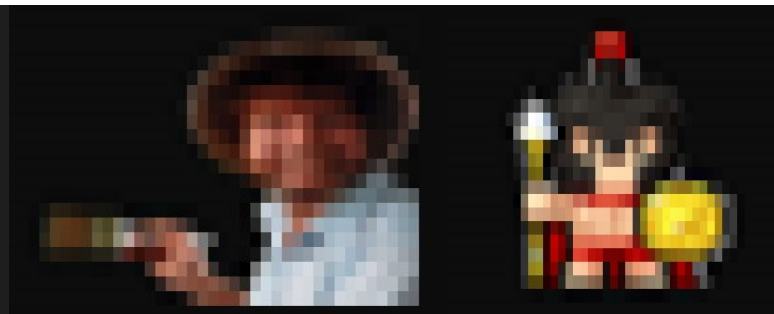
# 핵심기능 : Img2Ascii

class AsciiArt

```
{
    // 밝기 값에 따라 아스키 문자를 매핑하는 배열
    참조 0개
    private AsciiArt()
    {
    }

    참조 3개
    public static void Draw(string imagePath, int asciiWidth = 45)
    {
        Draw(imagePath, Console.CursorLeft, Console.CursorTop, asciiWidth);
    }

    참조 9개
    public static void Draw(string imagePath, int cursorX, int cursorY, int asciiWidth = 45)
    {
        imagePath = Directory.GetParent(Environment.CurrentDirectory).Parent.Parent.FullName + "\\ " + imagePath;
        using (Image<Rgb32> image = Image.Load<Rgb32>(imagePath))
        {
            // 아스키 아트로 변환
            ShowImageToAscii(image, cursorX, cursorY, asciiWidth);
        }
        Console.ResetColor();
    }
}
```



```
...// 이미지를 아스키 아트로 변환하는 함수
    참조 1개
    private static string ShowImageToAscii(Image<Rgb32> image, int cursorX, int cursorY, int asciiWidth)
    {
        int asciiHeight = (int)(image.Height / (double)image.Width * asciiWidth); // 이미지 비율 유지
        image.Mutate(x => x.Resize(asciiWidth, asciiHeight)); // 이미지 크기 조정

        StringBuilder asciiArt = new StringBuilder();

        for (int y = 0; y < image.Height; y++)
        {
            Console.SetCursorPosition(cursorX, cursorY + y);
            for (int x = 0; x < image.Width; x++)
            {
                Rgb32 pixelColor = image[x, y];

                // 각 픽셀의 밝기 계산
                int brightness = (int)((pixelColor.R + pixelColor.G + pixelColor.B) / 3.0);

                // 아스키 문자와 색상 정보 추가
                Console.Write($"{x1b[38;2;{pixelColor.R};{pixelColor.G};{pixelColor.B}]m{x1b[48;2;{pixelColor.R};{pixelColor.G};{pixelColor.B}]m");
            }
            return asciiArt.ToString();
        }
    }
```



# 핵심기능 : SoundPlayer

```
using System;
using System.IO;
using NAudio.Wave;

class SoundPlayer
{
    WaveOutEvent outputDevice;

    public SoundPlayer()
    {
    }

    public void Play(string filePath, float volume = 0.01f)
    {
        filePath = Directory.GetParent(Environment.CurrentDirectory).Parent.Parent.FullName + "\\
        var reader = new Mp3FileReader(filePath);
        var waveOut = new WaveOutEvent();
        waveOut.Volume = volume;
        waveOut.Init(reader);
        waveOut.Play();
    }

    2 references
    public void Stop()
    {
        using (outputDevice)
        {
            outputDevice.Stop();
        }
    }
}
```

설치 패키지  
NAudio

## 보인다 보여 들린다 들려

Words & Music by  
광종환





# 핵심기능 : Combat System

```
private void AttackTurn(Character turnOwner, Character attackTarget)
{
    AttackOrderInfo attackOrder = turnOwner.CreateAttackOrder(attackTarget);
    DefenseOrderInfo defenseOrder = attackTarget.CreateDefenseOrder(attackOrder);
    attackTarget.health -= defenseOrder.actualDamage;
    Console.WriteLine($"{turnOwner.name}이(가) {attackTarget.name}을(를) 공격했습니다.");
    Thread.Sleep(1000);
    if (attackTarget.health <= 0)
    {
        Console.WriteLine($"{attackTarget.name}이(가) 죽었습니다.");
    }
}
```

```
2 references
public AttackOrderInfo CreateAttackOrder(Character target)
{
    AttackOrderInfo attackOrderInfo = new AttackOrderInfo();
    attackOrderInfo.target = target;

    if(new Random().NextDouble() < this.stats.criticalRate)
    {
        attackOrderInfo.attackType = AttackType.CRITICAL;
        int minCriticalAttack = (int)Math.Ceiling(this.stats.minAttack * 1.6f);
        int maxCriticalAttack = (int)Math.Ceiling(this.stats.maxAttack * 1.6f);
        float damageTemp = new Random().Next(minCriticalAttack, maxCriticalAttack + 1);
    }
    else
    {
        attackOrderInfo.attackType = AttackType.NORMAL;
        attackOrderInfo.damage = new Random().Next(this.stats.minAttack, this.stats.maxAttack + 1);
    }

    return attackOrderInfo;
}
```

```
2 references
public DefenseOrderInfo CreateDefenseOrder(AttackOrderInfo attackOrderInfo)
{
    DefenseOrderInfo result = new DefenseOrderInfo();
    result.attackResult = attackOrderInfo;
    result.defenseAmount = (new Random().Next(this.stats.minArmor, this.stats.maxArmor + 1));
    result.actualDamage = attackOrderInfo.damage - result.defenseAmount;
    if (result.actualDamage <= 0)
    {
        result.actualDamage = 1;
    }

    return result;
}
```

```
private List<Character> CreateTurnOrder()
{
    List<Character> turnOrder = GetAllCharacters();
    for (int i = turnOrder.Count - 1; i >= 0; i--)
    {
        if (turnOrder[i].isDead)
        {
            turnOrder.RemoveAt(i);
        }
    }
    turnOrder.Sort((x, y) =>
    {
        Random rnd = new Random();
        int xOrder = rnd.Next(x.stats.minAgility, x.stats.maxAgility + 1);
        int yOrder = rnd.Next(y.stats.minAgility, y.stats.maxAgility + 1);
        return yOrder.CompareTo(xOrder);
    });
    return turnOrder;
}
```

```
while (isOnBattle)
{
    Console.Clear();
    ShowCharactersInfo();
    Character turnOwner = GetCurrentTurnOwner();
    if (turnOwner == null)
    {
        OnInitTurnCycle();
        turnOwner = GetCurrentTurnOwner();
    }
    if (turnOwner.isDead)
    {
        OnTurnEnd();
        continue;
    }
    DoTurn(turnOwner);
    Thread.Sleep(1000);
    if (IsAllPlayerCharacterDead())
    {
        isOnBattle = false;
        soundPlayer.Stop();
        ShowLose();
        break;
    }
    else if (IsAllMonsterDead())
    {
        isOnBattle = false;
        soundPlayer.Stop();
        ShowWin();
        break;
    }
    OnTurnEnd();
}
```



# 핵심기능 : Mercenary System

```
public class Mercenary : PlayerCharacter
{
    참조 7개
    public string jobName { get; set; } = "용병";

    참조 8개
    public int price { get; set; } = 0;

    참조 7개
    public string description { get; set; }

    참조 5개
    public Mercenary(string name, CharacterStats stats)
    {
        base(name, stats)
    }

    internal class MercenaryManager
    {
        this.maxExp = 0;

        private static readonly Lazy<MercenaryManager> lazy = new Lazy<MercenaryManager>(() => new MercenaryManager());
        참조 2개
        public static MercenaryManager instance { get { return lazy.Value; } }

        참조 8개
        public List<Mercenary> Mercenaries { get; }

        참조 1개
        private MercenaryManager()
        {
            Mercenaries = new List<Mercenary>();

            Mercenaries.Add(new SkilledArcher());
            Mercenaries.Add(new SkilledMagician());
            Mercenaries.Add(new SkilledMonk());
            Mercenaries.Add(new SkilledThief());
            Mercenaries.Add(new SkilledWarrior());
        }
    }
}
```

[상대 정보]

|                |           |
|----------------|-----------|
| [ 이기하대 ] - 몽크  |           |
| LV. 1 (0 / 10) |           |
| 체력             | : 150/150 |
| 공격력            | : 3 ~ 6   |
| 방어력            | : 7 ~ 14  |
| 민첩             | : 4 ~ 9   |
| 지령타율           | : 15%     |
| 회피율            | : 12%     |
| 골드             | : 1200 G  |

[고용한 용병 목록]

|               |               |               |           |
|---------------|---------------|---------------|-----------|
| [ 숙련된 궁수 ] -  | [ 숙련된 수도승 ] - | [ 숙련된 암살자 ] - |           |
| LV. 7 (0 / 0) | LV. 7 (0 / 0) | LV. 7 (0 / 0) |           |
| 체력            | : 200/200     | 체력            | : 170/170 |
| 공격력           | : 23 ~ 30     | 공격력           | : 20 ~ 30 |
| 방어력           | : 13 ~ 15     | 방어력           | : 8 ~ 10  |
| 민첩            | : 20 ~ 25     | 민첩            | : 24 ~ 30 |
| 지령타율          | : 25%         | 지령타율          | : 40%     |
| 회피율           | : 38%         | 회피율           | : 48%     |

[아무기]를 불러 돌아가기



# 핵심기능 : TargetingValidator

```
public abstract class TargetingValidator
{
    6 references
    public abstract bool IsValidFor(Character target);

    5 references
    protected virtual List<Character> ParseValidTargetCandidatesFrom(List<Character> targets)
    {
        List<Character> result = new List<Character>();
        foreach (Character target in targets)
        {
            if (IsValidFor(target))
            {
                result.Add(target);
            }
        }
        return result;
    }

    6 references
    public abstract List<Character> SelectTargetsFrom(List<Character> targets);
}
```

상속

```
public abstract class AiAction : TargetingValidator
{
    2 references
}
```

상속

```
public abstract class Skill : TargetingValidator
{
}
```

상



미안  
이미 만들었어ㅎ

**BLOCK!**

```
interface IConsumableItem : IItem
```

```
9 references
void OnUsed(Character character);
```

# 어려웠던 점

재사용.....



כג